Small Matrices

Detailed introduction to the dissertation reproducibility materials

Purpose. This document explains what the Small Matrices experiment is doing, why it is included in the dissertation, what the supplied Python files actually calculate, and what a reader should expect when reproducing the experiment. It is written so that a reader does not need prior knowledge of Python or numerical linear algebra to understand the experimental workflow.

1. The basic idea

The experiment is a controlled comparison of four numerical solvers: Direct, Conjugate Gradient (CG), LSQR and LSMR. All four are asked to solve the same underlying family of synthetic linear inverse problems. The purpose is not simply to find an answer to one matrix equation. Instead, the experiment changes selected properties of the problem and records how the four methods respond.

A useful way to think about the task is as follows. The program creates a matrix A, chooses a known reference vector x, and uses them to produce observations b. The solver is then given A and a noisy version of b, but it is not given the reference solution. Its job is to estimate x from the available information. Because the program knows the reference solution internally, it can later measure how accurate the estimated solution was.

In simple notation, the underlying relationship is A x = b. In the actual experiment the observations contain controlled noise, and positive regularisation values modify the problem so that the solver seeks a compromise between fitting the observations and avoiding unnecessarily large solution values.

2. Why this is called the Small Matrices experiment

The dissertation contains experiments at several scales. The Small Matrices section is the controlled baseline. The matrices are deliberately modest in size, with 50 columns and a row count ranging from 400 to 2100 in the matrix-size experiment. This makes direct solution computationally practical and allows the direct method to be compared with iterative methods under controlled conditions.

The point of this baseline is therefore methodological. It provides a relatively simple setting in which changes in matrix size, sparsity, spectral structure, conditioning and regularisation can be studied before considering the much larger problems used elsewhere in the dissertation.

3. What changes from experiment to experiment

The benchmark is organised as a series of one-factor sweeps. For each sweep, one experimental property is varied while the remaining properties are fixed at the values specified by the benchmark. This makes the resulting comparison easier to interpret: if solver behaviour changes as the condition parameter changes, for example, that change can be associated with the intended experimental factor rather than with several parameters changing simultaneously.

Factor	Values used	Fixed settings for that sweep
Matrix size	m = 400, 500, 1000, 1100, 2100	n = 50; sparsity 0; random pattern; condition 100; linear spectrum; lambda = 1e-6
Sparsity	0, 0.25, 0.5, 0.75, 0.9	m = 1000; n = 50; random pattern; condition 100; linear spectrum; lambda = 1e-6
Condition parameter	10, 100, 1000, 10000	m = 1000; n = 50; sparsity 0.5; random pattern; linear spectrum; lambda = 1e-6
Regularisation	0, 1e-8, 1e-6, 1e-4, 1e-2, 1e-1, 1	m = 1000; n = 50; sparsity 0.5; random pattern; condition 100; linear spectrum
Spectrum	clustered, exponential, linear, polynomial	m = 1000; n = 50; sparsity 0.5; random pattern; condition 100; lambda = 1e-6
Sparsity pattern	random, banded, block	m = 1000; n = 50; sparsity 0.75; condition 100; linear spectrum; lambda = 1e-6

Important: the regularisation values are written here in plain ASCII notation (for example, 1e-8) deliberately. This avoids font-dependent symbols in the PDF and preserves the exact numerical values used by the code.

4. How a synthetic matrix is constructed

The matrix is generated by the benchmark itself; no external matrix file is required. First, a sparse matrix structure is created. The requested sparsity controls how much of the matrix is retained. A sparsity of 0 means that no entries are deliberately removed, whereas a sparsity of 0.9 means that the target construction keeps roughly 10 percent of candidate entries. The random, banded and block patterns determine where those retained entries are allowed to occur.

The matrix is then scaled according to a selected spectral distribution. The code supports four spectral shapes: linear, exponential, polynomial and clustered. The condition parameter controls the range of spectral scales used in this construction. These values are therefore best described as **condition parameters**, not automatically as the exact condition number of the final sparse matrix.

This distinction matters. Once sparsification and the chosen structural pattern have been applied, the realised matrix can have a spectrum that differs from the idealised spectrum used to construct it. Consequently, a recorded condition parameter of 10000 does not guarantee that the final matrix has an exact numerical condition number of 10000.

5. What the sparsity patterns mean

- **Random:** candidate entries are retained at random according to the requested keep probability. This produces an unstructured sparse pattern.
- **Banded:** non-zero candidates are restricted to a band around the main diagonal and are then randomly retained according to the requested sparsity.
- **Block:** the matrix is divided into five broad row and column regions and candidate non-zeros are concentrated in corresponding diagonal blocks before the sparsity rule is applied.

6. How the observations are produced

After constructing A, the program creates a known reference vector x_true consisting of ones. It computes clean observations as A times x_true. A noise vector is then generated independently and normalised before being scaled so that its norm is 1 percent of the norm of the clean observations. The solver therefore receives noisy observations rather than the exact clean right-hand side.

This construction is useful for reproducibility because the reference solution remains known. The program can consequently report both how well a solver fits the observations and how close its recovered solution is to the known answer.

7. What the four solvers are doing

Direct. The direct method forms the normal-equation system explicitly and solves it using a sparse direct solver. With positive regularisation, the regularisation term is added to the normal equations.

Conjugate Gradient (CG). CG is applied to the regularised normal-equation operator. The implementation uses a LinearOperator, so the normal-equation matrix does not need to be explicitly formed as a separate dense matrix.

LSQR. LSQR solves the least-squares problem using its iterative Golub-Kahan process. In the regularised cases, the code supplies damping corresponding to the square root of the regularisation parameter.

LSMR. LSMR is another Golub-Kahan-based iterative least-squares method. It is evaluated under the same benchmark conditions and stopping limits as LSQR so that the comparison is meaningful.

8. What is recorded

Measure	Plain-English meaning
Runtime	How long the solver takes to produce its solution.
Iterations	How many iterative steps an iterative solver performs. The direct method is recorded as one solution step.
Convergence	Whether the solver reports successful termination under its stopping rules.
Relative residual	How large the remaining mismatch A x - b is relative to the observed data.
Relative normal residual	How small the normal-equation residual A^T(A x - b) is relative to A^T b.
Relative reconstruction error	How far the recovered solution is from the known reference solution x_true.
Objective	The squared residual plus the regularisation penalty: ||A x - b||^2 + lambda ||x||^2.

9. Repeated runs and random seeds

Each configuration is evaluated ten times. The benchmark uses a fixed base seed of 42 and derives separate pseudorandom streams for matrix construction and observation noise for each repeat and case. This means that the experiment does not depend on an uncontrolled stream of random numbers from the computer's current state.

The fixed seed makes the generated experimental procedure reproducible for the recorded implementation. It does not mean that every machine will produce identical timing values. Numerical-library versions, BLAS/LAPACK implementations, CPU architecture and system load can all affect runtime measurements.

10. The role of regularisation

Regularisation is controlled by the parameter lambda. The benchmark includes an unregularised case (lambda = 0) and six positive values: 1e-8, 1e-6, 1e-4, 1e-2, 1e-1 and 1. Positive regularisation adds a penalty for large solution vectors. In practical terms, the solver is no longer asked only to fit the observations; it is also encouraged to avoid unnecessarily large coefficients.

The experiment therefore asks whether increasing regularisation changes runtime, iteration count, convergence and reconstruction quality. The trade-off is important because a solution can fit noisy observations very closely without necessarily being the most accurate reconstruction of the underlying reference vector.

11. What the two Python files do

Small_Matrices.py is the benchmark and data-generation program. It constructs every requested case, generates the noisy problem, runs Direct, CG, LSQR and LSMR, records the metrics, repeats each configuration ten times by default, and writes CSV files containing the raw and aggregated results.

Plot_Small_Matrices.py is the analysis and visualisation program. It reads the generated result files and produces figures used to compare the solvers across the experimental factors. The plotting file does not replace the benchmark: it is a post-processing step that should be run after numerical results have been generated.

12. What reproducibility means for this repository

The repository is intended to preserve the experimental procedure rather than permanently store every generated CSV and PNG. The source files and documentation are the important reproducibility materials. A researcher can obtain the repository, create the documented Python environment, run the benchmark, and then run the plotting program to regenerate the numerical outputs and figures.

Removing generated outputs from the repository does not remove the experiment itself. It simply keeps the repository focused on source code, instructions and documentation rather than on machine-specific derived files.

13. What a reader should take away

The Small Matrices section is best understood as a controlled laboratory experiment for numerical solvers. The program creates synthetic problems with known answers, deliberately changes one property at a time, gives the

same class of problem to four solvers, repeats the tests, and records both computational cost and solution quality.

The experiment is therefore not intended to claim that one solver is universally best. Instead, it shows how solver behaviour depends on the structure and difficulty of the problem. Runtime, iterations, residuals and reconstruction error provide different views of that behaviour, and considering them together gives a more informative comparison than runtime alone.

14. Reproducibility summary

- The matrices are generated by the supplied code; no input matrix dataset is required.
- The benchmark uses four solvers: Direct, CG, LSQR and LSMR.
- The default benchmark performs ten repeats per configuration.
- The base random seed is 42 and the noise level is 0.01.
- The iterative tolerance is 1e-8 and the maximum iteration limit is 5000.
- The documented regularisation values are 0, 1e-8, 1e-6, 1e-4, 1e-2, 1e-1 and 1.
- The condition values are spectral-scaling parameters and should not automatically be reported as realised condition numbers after sparsification.
- Exact runtime values should not be expected to match across different machines.

In one sentence: this section creates small synthetic linear inverse problems, changes their size and numerical structure in a controlled way, solves them using four numerical methods, measures both speed and accuracy, and provides the code needed to reproduce the procedure.